

ÉCOLE D'INGÉNIEURS – CESI

Cycle Ingénieur A3 – Génie Logiciel

CAHIER DES CHARGES

Projet EasySave – Versions 1.1 et 2.0

Livrable 2 – EasySave v1.1 & v2.0

TUTEUR PEDAGOGIQUE : BENALI Smail

MEMBRES DU GROUPE :

- Olivier TCHATCHOUANG
- Chloe FOGUE
- Carelle TCHEMBENG
- Joseph MELINGUI

1. Présentation du projet

1.1 Contexte

EasySave 1.0 a été distribuée chez de nombreux clients. Suite à une enquête client, la direction de ProSoft a décidé de créer deux nouvelles versions pour répondre aux besoins exprimés :

- La version 1.1 (correction mineure) : ajoute la possibilité de choisir le format du fichier log (XML ou JSON). Elle reste en mode console.
- La version 2.0 (évolution majeure) : abandonne le mode console au profit d'une interface graphique, introduit le cryptage via CryptoSoft, le nombre de travaux illimité, et la détection du logiciel métier.

Un client important de ProSoft souhaitant que les fichiers log soient au format XML mais ne souhaitant pas installer la v2.0, la direction exige que la v1.1 soit livrée en même temps que la v2.0.

1.2 Objectifs du livrable 2

Le présent cahier des charges définit les spécifications fonctionnelles et techniques des versions 1.1 et 2.0 du logiciel EasySave. Ces deux versions constituent le Livrable 2.

1.3 Périmètre du livrable 2

Ce document couvre exclusivement EasySave versions 1.1 et 2.0. La version 3.0 fera l'objet d'un cahier des charges distinct (Livrable 3).

2. Contraintes générales du projet

Les contraintes générales définies dans le CDC v1.0 restent inchangées et s'appliquent également aux versions 1.1 et 2.0 :

2.1 Outils et environnement de travail

- IDE : Visual Studio 2022 ou version supérieure
- Versioning : GitHub (le tuteur/pilote doit être invité en tant que collaborateur)
- Éditeur UML : ArgoUML (recommandé)
- Tous les documents et codes sources doivent être gérés via ces outils

2.2 Langage et bibliothèque

- Langage : C#
- Bibliothèque cible : .NET 8.0
- Framework UI : WPF ou Avalonia (au choix de l'équipe) pour la v2.0

2.3 Qualité et maintenabilité du code

- L'ensemble des documents, lignes de code et commentaires doit être exploitable par les filiales anglophones (usage de l'anglais recommandé)
- Le nombre de lignes de code dans une fonction doit rester raisonnable (idéalement < 40 lignes)
- La redondance de code (copier-coller) est à proscrire (principe DRY)
- Les conventions de nommage C# doivent être respectées
- La documentation utilisateur doit tenir en une seule page
- Une release note est obligatoire pour chaque version

3. Tableau de comparaison des versions

Le tableau ci-dessous récapitule l'évolution fonctionnelle entre les trois versions d'EasySave :

Fonction	Version 1.0	Version 1.1	Version 2.0
Interface Graphique	Console	Console	Graphique (WPF/Avalonia)
Multi-langues	Anglais et Français	Anglais et Français	Anglais et Français
Travaux de sauvegarde	Limité à 5	Limité à 5	Illimité
Fichier Log journalier	Oui	Oui	Oui + temps de cryptage
DLL EasyLog	Oui	Oui	Oui
Fichier État temps réel	Oui	Oui	Oui
Format du log	JSON uniquement	JSON ou XML (choix utilisateur)	JSON ou XML (choix utilisateur)
Type de sauvegarde	Complète / Différentielle	Complète / Différentielle	Complète / Différentielle
Arrêt si logiciel métier	Non	Non	Oui
Ligne de commande	Oui	Oui	Identique v1.0
Cryptage CryptoSoft	Non	Non	Oui

4. Exigences du système – EasySave v1.1

La version 1.1 est une évolution mineure de la v1.0. Elle reste une application console .NET 8 avec toutes les fonctionnalités existantes, et introduit uniquement la gestion du format du fichier log.

4.1 Exigences fonctionnelles – v1.1

Catégorie	Description
Format de log	L'utilisateur peut choisir le format du fichier log : JSON ou XML, depuis les paramètres généraux.
Persistance du choix	Le choix du format de log est persisté dans le fichier de configuration config.json.
Compatibilité log XML	Le fichier log XML doit contenir les mêmes informations que le log JSON de la v1.0.
Rétrocompatibilité	La version 1.1 reste compatible avec tous les comportements de la v1.0 (console, 5 travaux max, bilingue).

4.2 Détail des fonctionnalités v1.1

4.2.1 Choix du format de log

L'utilisateur peut choisir entre JSON et XML comme format de sortie pour le fichier log journalier, directement depuis le menu interactif ou dans les paramètres.

- En JSON : comportement identique à la v1.0 (format yyyy-MM-dd.json)
- En XML : fichier au format yyyy-MM-dd.xml contenant les mêmes champs que le log JSON
- Le choix est persisté dans config.json

5. Exigences du système – EasySave v2.0

La version 2.0 est une évolution majeure. Elle abandonne le mode console au profit d'une interface graphique et introduit plusieurs nouvelles fonctionnalités.

5.1 Exigences fonctionnelles – v2.0

Catégorie	Description	Priorité
Interface Graphique	L'application doit disposer d'une interface graphique (WPF ou Avalonia). Le mode console est abandonné.	Haute
Travaux illimités	Le nombre de travaux de sauvegarde n'est plus limité à 5. L'utilisateur peut en créer autant que nécessaire.	Haute

Catégorie	Description
Cryptage CryptoSoft	Le logiciel doit crypter les fichiers en utilisant le logiciel externe CryptoSoft. Seuls les fichiers dont l'extension est définie par l'utilisateur dans les paramètres sont cryptés.
Paramétrage extensions	L'utilisateur définit dans les paramètres généraux les extensions de fichiers à crypter (ex : .txt, .pdf).
Log temps de cryptage	Le fichier log journalier doit contenir une information supplémentaire : le temps de cryptage en ms (0 = pas de cryptage, >0 = durée, <0 = code erreur).
Logiciel métier	Si un logiciel métier défini par l'utilisateur est détecté en cours d'exécution, le démarrage d'une nouvelle sauvegarde est interdit.
Arrêt en cours	Si le logiciel métier est détecté pendant une sauvegarde séquentielle, la sauvegarde du fichier en cours est terminée avant l'arrêt.
Log arrêt métier	L'arrêt causé par la détection du logiciel métier doit être consigné dans le fichier log.
Paramétrage logiciel métier	L'utilisateur définit le nom du logiciel métier dans les paramètres généraux. (Ex : calculatrice pour les démos.)
Format log XML ou JSON	Comme en v1.1, l'utilisateur choisit le format du log (JSON ou XML) dans les paramètres généraux.
Ligne de commande	L'exécution via ligne de commande reste identique à la v1.0 (plage, liste de travaux).
Bilingue FR/EN	L'interface graphique reste bilingue FR/EN avec détection automatique de la langue système.

5.2 Détail des fonctionnalités v2.0

5.2.1 Interface Graphique

L'application abandonne le mode console. Elle doit être développée avec WPF ou un framework alternatif tel qu'Avalonia. L'interface doit permettre de :

- Créer, modifier, supprimer et exécuter des travaux de sauvegarde
- Accéder aux paramètres généraux (format log, extensions à crypter, logiciel métier)
- Visualiser l'état des sauvegardes en temps réel

5.2.2 Travaux de sauvegarde illimités

Le nombre de travaux de sauvegarde n'est plus limité à 5. La liste des travaux est stockée dans le fichier de configuration config.json sous forme de tableau dynamique.

5.2.3 Cryptage via CryptoSoft

Le logiciel doit être capable de crypter les fichiers en utilisant le logiciel CryptoSoft (développé dans le cadre du PBL 4). Le cryptage s'applique selon les règles suivantes :

- Seuls les fichiers dont l'extension figure dans la liste définie par l'utilisateur dans les paramètres généraux sont cryptés
- CryptoSoft est appelé comme processus externe lors de la copie de chaque fichier éligible
- Le temps de cryptage (en ms) est enregistré dans le log journalier :
 - 0 : pas de cryptage (extension non ciblée)
 - > 0 : durée du cryptage en millisecondes
 - < 0 : code erreur retourné par CryptoSoft

5.2.4 Logiciel Métier

Si la présence d'un logiciel métier est détectée en cours d'exécution, le comportement est le suivant :

- Avant démarrage d'une sauvegarde : le lancement du travail est interdit
- Pendant une sauvegarde séquentielle : la copie du fichier en cours est terminée, puis la sauvegarde s'arrête
- L'arrêt est consigné dans le fichier log
- L'utilisateur définit le logiciel métier dans les paramètres généraux (ex : calculatrice pour les démonstrations)

5.2.5 Format du log

Comme en v1.1, l'utilisateur choisit le format du log (JSON ou XML) dans les paramètres généraux. Ce choix est persisté dans config.json.

5.2.6 Fonctionnalités héritées de la v1.0

Les fonctionnalités suivantes sont conservées à l'identique dans la v2.0 :

- Sauvegarde complète et différentielle
- Fichier état temps réel (state.json)
- DLL EasyLog (rétrocompatible)
- Bilingue FR/EN avec détection automatique de la langue système

- Support des chemins locaux, externes et réseau (UNC)
- Exécution via ligne de commande (identique v1.0)

6. Exigences non fonctionnelles

Catégorie	Description
Performance	Le temps de copie et de cryptage doit rester optimal. L'écriture des logs ne doit pas bloquer la sauvegarde.
Fiabilité	Aucun crash brutal. Les erreurs de transfert ou de cryptage sont consignées sans interrompre les autres fichiers.
Maintenabilité	Code sans duplication (DRY), fonctions courtes (< 40 lignes), conventions C# respectées, commentaires en anglais.
Extensibilité	Architecture MVVM préservée. L'ajout de nouvelles fonctionnalités ne doit pas nécessiter de refonte.
Compatibilité	Compatible disques locaux, externes et réseau (UNC). Aucune dépendance à un chemin fixe.
Format fichiers	Tous les fichiers JSON et XML doivent être bien indentés et lisibles directement dans Notepad.
Rétrocompatibilité DLL	EasyLog.dll v2.0 doit rester rétrocompatible avec la v1.0.
Sécurité	Le cryptage CryptoSoft ne doit s'appliquer qu'aux extensions explicitement autorisées par l'utilisateur.

7. Modélisation orientée objet & Architecture

7.1 Architecture en couches

L'architecture en couches définie en v1.0 est conservée et enrichie pour la v2.0 :

- EasySave.Core : Logique métier pure. Intègre désormais la gestion du logiciel métier et l'appel à CryptoSoft.
- EasySave.Infrastructure : Accès aux ressources externes (fichiers, JSON/XML, CryptoSoft process).
- EasySave.UI (nouveau) : Couche de présentation graphique (WPF ou Avalonia). Remplace EasySave.ConsoleApp pour la v2.0. La ConsoleApp est maintenue pour la v1.1.
- EasyLog : DLL indépendante, enrichie pour supporter le champ temps de cryptage.

7.2 Pattern MVVM

La v2.0 adopte le pattern MVVM (Model-View-ViewModel) pour séparer la logique métier de l'interface graphique :

- Model : entités du Core (BackupJob, BackupResult, etc.)
- ViewModel : expose les données et commandes à la vue (INotifyPropertyChanged)
- View : fichiers XAML ou Avalonia (interface graphique)

7.3 Principes de conception

- Principes SOLID appliqués : SRP, OCP, DIP
- Design Pattern Strategy conservé pour les types de sauvegarde
- Nouveau Pattern : Observer/Event pour la détection du logiciel métier en temps réel
- Gestion de versions GitHub : branches main, develop, feature/xxx, bugfix/xxx

8. Critères de validation

Le projet Livrable 2 sera considéré comme conforme si les critères suivants sont satisfaits :

Catégorie	Critère d'évaluation	Poids
Respect des contraintes ProSoft	Outils (VS, .NET 8, GitHub, ArgoUML), conventions de code	15%
Gestion de projet	Qualité du Git (branching, commits, historique)	15%
Qualité du code	Propreté, DRY, maintenabilité, conventions C#	20%
Architecture & Conception	Séparation MVVM, extensibilité, SOLID	15%
Fonctionnalités	Respect du cahier des charges v2.0 et v1.1	20%
Documentation	Manuel utilisateur, Release Notes, README	10%
Tests & Robustesse	Gestion erreurs, cryptage, logiciel métier, cas limites	5%

9. Livrables attendus

9.1 Code source et exécutables

- Code source complet versionné sur le dépôt GitHub
- Application graphique compilée (v2.0) avec DLL EasyLog
- Application console compilée (v1.1) avec DLL EasyLog

9.2 UML

- Diagrammes de classes mis à jour (v1.1 et v2.0)
- Diagramme de cas d'utilisation mis à jour
- Diagrammes de séquence (cryptage, logiciel métier)
- Livrés 24h avant le jour du livrable (la veille)

9.3 Documentation

- User Manual : manuel d'utilisation en une page (FR/EN) – v1.1 et v2.0
- Technical Doc : emplacements par défaut, prérequis, configuration minimale
- Release Note v1.1 et Release Note v2.0

10. Calendrier

Échéance	Action
Lendemain Livrable 1	Mise à disposition du CDC v2.0 et v1.1
Jour 3 après L1	Création de l'environnement de travail v2 (nouvelle branche GitHub)
Veille Livrable 2	Livraison des diagrammes UML (mis à jour)
Jour Livrable 2	Réception du Livrable 2 (EasySave v2.0 + v1.1) et documentations associées

11. Glossaire

Terme	Définition
Travail de sauvegarde	Unité de configuration définissant une opération de copie entre un répertoire source et un répertoire cible.
Sauvegarde complète	Copie intégrale de tous les fichiers du répertoire source vers le répertoire cible.
Sauvegarde différentielle	Copie uniquement des fichiers modifiés depuis la dernière sauvegarde complète.
Format UNC	Universal Naming Convention : format de chemin réseau (ex : \\serveur\dossier\fichier).
EasyLog.dll	Bibliothèque dynamique développée par l'équipe, dédiée à la journalisation des actions de sauvegarde.
State.json	Fichier JSON unique enregistrant en temps réel l'état de progression de chaque travail de sauvegarde.
CryptoSoft	Logiciel tiers de cryptage utilisé par EasySave v2.0 pour chiffrer les fichiers selon les extensions définies.
Logiciel Métier	Application définie par l'utilisateur dont la présence en mémoire déclenche l'arrêt ou l'interdiction de sauvegarde.
MVVM	Model-View-ViewModel : pattern architectural utilisé pour séparer la logique métier de l'interface graphique.
WPF / Avalonia	Frameworks de développement d'interfaces graphiques pour applications .NET.
DLL	Dynamic Link Library – bibliothèque de fonctions réutilisables chargée dynamiquement par le programme.
.NET 8	Version de la plateforme de développement Microsoft utilisée pour ce projet.